

Lab 2 – Solutions

September 30, 2024

This lab deals with the numerical approximation of the zeros of a real valued function of one variable, that is

given $f : \mathcal{I} = (a, b) \subset \mathbb{R} \rightarrow \mathbb{R}$, find $\alpha \in \mathbb{R}$ such that $f(\alpha) = 0$.

Methods for the numerical approximations of a zero of f are usually iterative; the aim is to generate a sequence of values $x^{(k)}$ such that

$$\lim_{k \rightarrow \infty} x^{(k)} = \alpha$$

1 Bisection method

Theorem 2.1 (Bolzano's theorem). Let $f \in \mathcal{C}^0([a, b])$ with $f(a)f(b) < 0$, then $\exists \alpha \in (a, b)$ s.t. $f(\alpha) = 0$.

Moreover, if f is monotone in $[a, b]$ (e.g., $f \in \mathcal{C}^1([a, b])$ and f' has constant sign), then α is unique.

Method 2.1 (Bisection method). Let $x_{left}^{(0)} = a$, $x_{right}^{(0)} = b$; at step $k \in \mathbb{N}$ compute the midpoint $\bar{x}^{(k)} = (x_{left}^{(k)} + x_{right}^{(k)})/2$ and $f(\bar{x}^{(k)})$:

- if $f(x_{left}^{(k)})f(\bar{x}^{(k)}) < 0$, set $x_{left}^{(k+1)} = x_{left}^{(k)}$ and $x_{right}^{(k+1)} = \bar{x}^{(k)}$;
- else set $x_{left}^{(k+1)} = \bar{x}^{(k)}$ and $x_{right}^{(k+1)} = x_{right}^{(k)}$.

and stop when the number of iteration $n = N_{Max} = \left\lceil \log_2 \left(\frac{b-a}{tol} \right) \right\rceil = \left\lceil \frac{\log \frac{b-a}{tol}}{\log 2} \right\rceil$

Comments

- Global convergence, but not monotone decreasing of the error
- slow convergence
- Need to provide upper and lower bounds

Exercise 2.1. Consider the following function

$$f(x) = x^3 - (2+e)x^2 + (2e+1)x + (1-e) - \cosh(x-1), x \in [0.5, 5.5].$$

- Plot the function f and determine two intervals that contain its roots (use the Matlab command `grid on`).
- For which roots the bisection method can be used? Compute the number of needed iterations for the bisection method to converge with a tolerance of 10^{-3} , when the interval $[3, 5]$ is chosen as starting interval.
- Implement the bisection method: `function [x,x_iter]=bisection(f,a,b,tol)` where x is the solution, x_iter is the vector of the approximations at each iteration, f is the function, defined as handle function, a, b are the end points of the interval, tol is the required tolerance.
- Employ the bisection method to approximate the roots with a tolerance equal to 10^{-3} .
- Plot the evolution of the error with the iterations.

Solution Exercise 2.1.

```
function [xi, x_iter] = bisection(f, a, b, tol)
%BISECTION Find a root of the equation  $f(x) = 0$  in the interval  $[a,b]$ 
% using bisection method. The interval limits must be such that  $f(a)*f(b)<0$ .
%
% [xi, x_iter] = BISECTION(f, a, b, tol)
%
% Inputs : f      = function handle to the function  $f(x)$ 
%          a      = left bound
%          b      = right bound
%          tol    = requested tolerance
% Output :
%          xi     = approximation of the root
%          x_iter = vector of the approximations of the root at each step

% Number of necessary iterations to match the requested tolerance

max_iterb = ceil( log((b - a)/tol)/log(2) );
if f(a)*f(b) >= 0
    error('The sign of the function at the interval endpoints must be different\b');
elseif f(a)==0
    xi = a; x_iter = a;
    return;
elseif f(b)==0
    xi = b; x_iter = b;
    return;
end
x_iter=zeros(max_iterb, 1);
for iter = 1:max_iterb
    x_iter(iter) = a + (b-a)/2;
    f_iter(iter) = f(x_iter(iter));

    if ( f(b)*f_iter(iter) < 0 )
        a = x_iter(iter);
    elseif ( f(a)*f_iter(iter) < 0 )
        b = x_iter(iter);
    else %  $f(x) = 0$ 
        break;
    end
end
xi = x_iter(end);

end
```

File bisection.m

```
% EXERCISE 2.1
%% closes all figures whose handles are visible
close all
clear all
clc

%% PLOT THE FUNCTION
f= @(x) x.^3-(2 + exp(1))*x.^2 + (2*exp(1) + 1)*x + (1 - exp(1)) - cosh(x - 1);
%f= @(x) x.^3-(2 + exp(1))*x.^2 + (2*exp(1) + 2)*x + (1 - exp(1)) - sinh(x - 1);

a=0.5; b=5.5;
x_plot=linspace(a,b,1000);
figure
plot(x_plot,f(x_plot),'LineWidth', 2);
% add a grid to the plot
grid on
xlim([a, b])
xlabel("x",'FontSize', 18)
ylabel("f(x)", 'FontSize', 18)
str=sprintf('Function f in [%d, %d]', a, b);
title(str, 'FontSize', 15)
set(gca,'FontSize', 16)
```

```

set(gca,'LineWidth', 1.5)
% pause the execution
% pause

% from the plot two different roots can be identified. The bisection method,
% nevertheless, can be applied only to the root on the right: the one
% belonging to the interval [3,5]. Actually, in the other case, there isn't
% any interval such that the first derivative has constant sign and the
% root belongs to this interval. Therefore the Bolzano theorem hypothesis
% are not respected in this case.

%%
% FIND THE ROOT (BISECTION METHOD)
% The interval is [3,5]
a=3;
b=5;
tol=1.e-3;

[x,x_iter]=bisection(f,a,b,tol);
%bisection_plot(f, a, b, tol)
str=fprintf("The root in the interval [%d, %d] is x=%f. \n", a, b, x);

%%
% PLOT THE ITERATIONS OF THE FUNCTION

x_plot=linspace(a,b,1000);
figure
plot(x_plot,f(x_plot), 'LineWidth', 2);
str=sprintf("Iterations required to reach the tolerance: %f", length(x_iter));
title(str,'FontSize', 15)
grid on
hold on
%disp('Press any key...');
for ii=1:length(x_iter)
    plot(x_iter(ii),f(x_iter(ii)), 'r*')
    xlabel("x", 'FontSize', 18)
    ylabel("f(x)", 'FontSize', 18)
    pause(3)
end

%% PLOT THE ERROR
figure

for ii=1:length(x_iter)
    plot((ii),abs(x_iter(ii)-x), 'r*')
    drawnow
    hold on
    xlabel("iter", 'FontSize', 18)
    ylabel("err", 'FontSize', 18)
    pause(2)
end

```

2 Newton method

Method 2.2 (Newton method). *Newton method consists in approximating the zero of $f(x)$ with the sequence*

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}, k \geq 0, f'(x^{(k)}) \neq 0$$

Theorem 2.2. *If $f \in C^2([a,b])$ and $f'(x) \neq 0$ in an open interval containing α , then $\exists \delta > 0$ s.t. $\forall x^{(0)} : |x^{(0)} - \alpha| < \delta$ the Newton method converges **quadratically** to α . Moreover, if in addition $f(a)f(b) < 0$ and $f''(x)$ has constant sign in $[a,b]$, the method converges for any $x^{(0)} \in [a,b]$ s.t. $f(x^{(0)})f''(x^{(0)}) > 0$.*

Comments

- conditional convergence (dependence on the initial guess $x^{(0)}$)
- α simple zero ($f'(\alpha) \neq 0$): quadratic conditional convergence
- α zero of multiplicity m ($f^{(i)}(\alpha) = 0, i = 0, \dots, m-1, f^{(m)}(\alpha) \neq 0$): linear conditional convergence

Convergence order of a scheme $\{x^{(k)}\}_{k \in \mathbb{N}}$ approximating a zero α

$$p = \lim_{k \rightarrow \infty} \frac{\log \frac{e^{(k+1)}}{e^{(k)}}}{\log \frac{e^{(k)}}{e^{(k-1)}}},$$

with $e^{(k)} = |x^{(k)} - \alpha|$ error at the k -th iteration.

Exercise 2.2. Consider the following function in the interval $[-0.5, 1.5]$

$$f(x) = \sin(x)(1-x)^2.$$

- Plot f in order to find some intervals containing the roots.
- Implement the Newton method by using a stopping criterion based on the error estimator $|x^k - x^{k-1}|$. The signature of the function is: `function [x,x_iter]=newton(f,df,x0,tol,Nmax)` where x is the approximate, x_iter is the vector of the approximations at each iteration, f , df are the function and its first derivative, defined as handle functions, $x0$ is the initial guess, tol is the tolerance demanded by user and $Nmax$ is the maximum number of allowed iterations.
- Use Newton method to find the roots with a tolerance equal to 10^{-6} , by considering as initial guess $x_0 = 0.3$ and $x_0 = 0.5$.
- Compute an estimate of the convergence rate. Is it the expected one?

Solution Exercise 2.2.

```
function [x,x_iter]=newton(f,df,x0,tol,Nmax)

ii=1;
err=1+tol;
% or we can set
% err = inf;
x_iter(ii)=x0;

while ii<=Nmax && err>tol
    f_x=f(x_iter(ii));
    df_x= df(x_iter(ii));
    if(abs(df_x) < 1e-8) % check if the derivative (denominator) is zero (or very small)
        fprintf("The derivative of f in %d approaches or is equal to zero", x_iter(ii))
        break;
    end

    %delta=-df_x\f_x;
    %x_iter(i+1)=x_iter(i)+delta;
    %err=abs(delta);

    x_iter(ii+1)=x_iter(ii)-f_x/df_x;
    err=abs(x_iter(ii+1)-x_iter(ii));

    ii=ii+1;
    if(ii==Nmax)
        fprintf("Maximum number of iteration reached. Newton did not converge!\n")
    end
end
```

```
x=x_iter(end);
end
```

File newton.m

```
clear all
close all
clc
%% Point 1
% plot the function f and its two first derivatives
x = linspace (-2, 2, 1001);
fun = @(x) exp (x) - x.^2 - sin (x) - 1;

figure (1)
plot (x, fun (x), 'r', 'LineWidth', 2)
hold on, grid on
plot (x, zeros (size (x)), 'b')
xlabel ('x')
ylabel ('y')
title ('y = f (x)')

figure (2)
dfun = @(x) exp (x) - 2*x - cos (x);
plot (x, dfun (x), 'r', 'LineWidth', 2)
hold on, grid on
plot (x, zeros (size (x)), 'b')
xlabel ('x')
ylabel ('y')
title ('y = df (x)')

figure (3)
d2fun = @(x) exp (x) - 2 + sin (x);
plot (x, d2fun (x), 'r', 'LineWidth', 2)
hold on, grid on
plot (x, zeros (size (x)), 'b')
xlabel ('x')
ylabel ('y')
title ('y = d2f (x)')

% x=0 multiplicity m=2
% x=1.2797 simple root
%% Point 2
% Newton Method for the root with multiplicity m=2
x0 = 0.1;
nmax = 100;
tol = 1e-6;
disp ('Newton Method for the root with multiplicity m=2')
[x, xvect] = newton (fun,dfun, x0,tol, nmax);

% stima ordine e fattore di convergenza
[p, c] = stimap (xvect);

% Grafico dell'errore in funzione delle iterazioni
x1 = 0;
err_exact = abs (xvect - x1);
figure (4)
semilogy (1:numel(err_exact ), err_exact , 'b*')
grid on
xlim ([1, numel(err_exact )])
xlabel ('iterazioni')
ylabel ('errore')
title ('Convergenza lineare per Newton su x = 0')

%% Punto 3
% Metodo di Newton per lo zero semplice
x0 = 1.2;
disp ('Metodo di Newton per lo zero semplice')
[x, xvect] = newton (fun,dfun, x0,tol, nmax);

% stima ordine e fattore di convergenza
```

```
[p, c] = stimap (xvect);

% Grafico dell'errore in funzione delle iterazioni
x2 = 1.279701331000996;
errore_vero = abs (xvect - x2);
figure (5)
semilogy (1:numel(errore_vero), errore_vero, 'b*')
grid on
xlabel ('iterazioni')
ylabel ('errore')
title ('Convergenza quadratica per Newton su x = 1.27..')
```

Exercise 2.3. Consider the following function in the interval $[-2, 2]$

$$f(x) = \exp(x) - x.^2 - \sin(x) - 1;$$

- Plot f and its two first derivatives. How many roots does the function have? What is the multiplicity of each root?
- Find the roots of the function using the Newton method with a tolerance of 10^{-6} and initial guess $x^{(0)} = 0.1$ for the root with $m > 1$ and $x^{(0)} = 1.2$ for the simple root. Compute the error considering the exact solutions are $x_1 = 0$ and $x_2 = 1.2797$.
- Compute an estimate of the convergence rate. Is it the expected one?

Solution Exercise 2.3.

```
clear all
close all
clc
%% Point 1
% plot the function f and its two first derivatives
x = linspace (-2, 2, 1001);
fun = @(x) exp (x) - x.^2 - sin (x) - 1;

figure (1)
plot (x, fun (x), 'r', 'LineWidth', 2)
hold on, grid on
plot (x, zeros (size (x)), 'b')
xlabel ('x')
ylabel ('y')
title ('y = f (x)')

figure (2)
dfun = @(x) exp (x) - 2*x - cos (x);
plot (x, dfun (x), 'r', 'LineWidth', 2)
hold on, grid on
plot (x, zeros (size (x)), 'b')
xlabel ('x')
ylabel ('y')
title ('y = df (x)')

figure (3)
d2fun = @(x) exp (x) - 2 + sin (x);
plot (x, d2fun (x), 'r', 'LineWidth', 2)
hold on, grid on
plot (x, zeros (size (x)), 'b')
xlabel ('x')
ylabel ('y')
title ('y = d2f (x)')

% x=0 multiplicity m=2
% x=1.2797 simple root
%% Point 2
% Newton Method for the root with multiplicity m=2
x0 = 0.1;
nmax = 100;
tol = 1e-6;
disp ('Newton Method for the root with multiplicity m=2')
[x, xvect] = newton (fun,dfun, x0,tol, nmax);
```

```

% stima ordine e fattore di convergenza
[p, c] = stimap (xvect);

% Grafico dell'errore in funzione delle iterazioni
x1 = 0;
err_exact = abs (xvect - x1);
figure (4)
semilogy (1:numel(err_exact ), err_exact , 'b*')
grid on
xlim ([1, numel(err_exact )])
xlabel ('iterazioni')
ylabel ('errore')
title ('Convergenza lineare per Newton su x = 0')

%% Punto 3
% Metodo di Newton per lo zero semplice
x0 = 1.2;
disp ('Metodo di Newton per lo zero semplice')
[x, xvect] = newton (fun,dfun, x0,tol, nmax);

% stima ordine e fattore di convergenza
[p, c] = stimap (xvect);

% Grafico dell'errore in funzione delle iterazioni
x2 = 1.279701331000996;
errore_vero = abs (xvect - x2);
figure (5)
semilogy (1:numel(errore_vero), errore_vero, 'b*')
grid on
xlabel ('iterazioni')
ylabel ('errore')
title ('Convergenza quadratica per Newton su x = 1.27..')

```

3 Bisection - Newton methods

Theorem 2.3. *If $f \in C^2([a, b])$ and $f'(x) \neq 0$ in an open interval containing ξ , then $\exists \delta > 0$ s.t. $\forall x^{(0)} : |x^{(0)} - \xi| < \delta$ the Newton method converges quadratically to ξ .*

The convergence is guaranteed only if the initial guess $x^{(0)}$ is close enough to the root ξ , and for this reason the Newton method is a **locally** convergent method. A simple solution to overcome this issue consists in employing the bisection method to predict the initial guess $x^{(0)}$, as shown in the next exercise.

Exercise 2.4. Consider the following function in the interval $[-1, 6]$

$$f(x) = \arctan \left[7 \left(x - \frac{\pi}{2} \right) \right] + \sin \left[\left(x - \frac{\pi}{2} \right)^3 \right].$$

- Plot f in order to find an interval containing a root. What is the multiplicity of the root?
- Use the Newton method to find the root with a tolerance of 10^{-10} and initial guess $x^{(0)} = 1.5$. Compute the error.
- Use the Newton method to find the root with a tolerance of 10^{-10} and initial guess $x^{(0)} = 4$. Compute the error.
- If possible, apply the bisection method on the interval $[a, b] = [-1, 6]$ and tolerance $\frac{b-a}{2^{30}}$. Compute the error.
- Write a function `bisection_newton.m` to find ξ using the Newton method starting from an initial guess obtained after few iterations of a bisection method. Test with $[a, b] = [-1, 6]$, 5 iterations of the bisection method and tolerance 10^{-10} for the Newton method.

Solution Exercise 2.4.

```

function [xi, x_iter_bisection, x_iter_newton] = bisection_newton(f, df, a, b, tol_bisection,
tol_newton, maxit_newton)
%NEWTON Find a root of the equation  $f(x) = 0$  using the Newton method, starting from an
initial guess obtained by few iterations of a bisection method.
%
% [xi, x_iter] = BISECTION_NEWTON(f, df, a, b, tol_bisection, tol_newton, maxit_newton,
multiplicity)
%
% Inputs : f = function handle to the function  $f(x)$ 
%          df = function handle to the derivative of the function  $f(x)$ 
%          a = left bound
%          b = right bound
%          tol_bisection = requested tolerance for the bisection method
%          tol_newton = requested tolerance for the Newton method
%          maxit_newton = maximum number of iterations for the Newton method
%
% Output :
%          xi = approximation of the root
%          x_iter = vector of the approximations of the root at each step
% margin: return the number of input values regarding the current
% function

[xi_bisection, x_iter_bisection] = bisection(f, a, b, tol_bisection);
[xi_newton, x_iter_newton] = newton(f, df, xi_bisection, tol_newton, maxit_newton);

xi = xi_newton;

end
% EXERCISE 2.4

a = -1;
b = 6;
f = @(x) atan(7*( x - pi/2)) + sin((x-pi/2).^3);
rootfinding_function_plot(f, a, b, true);

% The function is null at  $xi = \{pi\}/\{2\}$ .

xi_ex = pi/2;
df = @(x) 7 ./ ( 1 + 49 * ( x-pi/2 ).^2 ) + 3 * (x-pi/2).^2 .* cos( (x-pi/2).^3 );
df(xi_ex);
% The multiplicity of the root is one since the derivative of  $f$  at  $xi = \{pi\}/\{2\}$  is different
from zero.

x0 = 1.5;
tol = 1e-10;
maxit = 1000;
[xi1, x_iter1] = newton(f, df, x0, tol, maxit);
xi1
iter1 = numel(x_iter1) - 1
err1 = abs( xi1 - xi_ex)
% Newton method converges to  $xi$  within the prescribed tolerance in very few iterations.

x0 = 4;
tol = 1e-10;
maxit = 1000;
[xi2, x_iter2] = newton(f, df, x0, tol, maxit);
xi2
iter2 = numel(x_iter2) - 1
err2 = abs( xi2 - xi_ex)
% Newton method does not converge to  $xi$ , and stops because of the maximum number of iterations
has been reached.
%In this case the initial guess is not close enough to the root  $xi$  and the local convergence
result does not apply.

tol = (b-a)/(2^30);
[xi3, x_iter3] = bisection(f, a, b, tol);
xi3
iter3=numel(x_iter3)
err3=abs( xi3 - xi_ex)
%%

```

```

% setting the bisection tolerance as  $(b-a)/2^k$  implies that the maximum
% number of iterations of bisection is set to k

tol_bisection = (b-a)/(2^5);
tol_newton = 1e-10;
maxit_newton = 1000;
[xi4, x_iter4_bisection, x_iter4_newton] = bisection_newton(f, df, a, b, tol_bisection,
tol_newton, maxit_newton);
xi4
iter4_bisection=numel(x_iter4_bisection)
iter4_newton=numel(x_iter4_newton)
err4 = abs( xi4 - xi_ex)

```